

ESAME DI FONDAMENTI DI INFORMATICA T-2 del 14/2/2024

Proff. E. Denti – R. Calegari – A. Molesini

Tempo a disposizione: 3h30

NOME PROGETTO ECLIPSE: CognomeNome-matricola (es. RossiMario-0000123456)
NOME CARTELLA PROGETTO: CognomeNome-matricola (es. RossiMario-0000123456)
NOME ZIP DA CONSEGNARE: CognomeNome-matricola.zip (es. RossiMario-0000123456.zip)
NOME JAR DA CONSEGNARE: CognomeNome-matricola.jar (es. RossiMario-0000123456.jar)

Si devono consegnare DUE FILE: l'intero progetto Eclipse e il JAR eseguibile

Si ricorda che compiti *non compilabili o palesemente lontani da 18/30* NON SARANNO CORRETTI e causeranno la verbalizzazione del giudizio "RESPINTO"

Dentinia Trasporti ha richiesto di sviluppare un'app per permettere agli utenti di sapere *l'orario di passaggio della prossima corsa* in una qualunque fermata della sua rete di tram.

DESCRIZIONE DEL DOMINIO DEL PROBLEMA

La rete di tram di Dentinia è costituita da varie **linee** "da punto a punto" (non esistono linee circolari): tutte le linee sono percorse nei due sensi e tutte le fermate sono servite sia dalle corse "di andata" (dal capolinea A al capolinea B) sia da quelle "di ritorno" (dal capolinea B al capolinea A).

Ogni **fermata** è caratterizzata dal suo *nome* e dalla *distanza in minuti dal capolinea iniziale*.

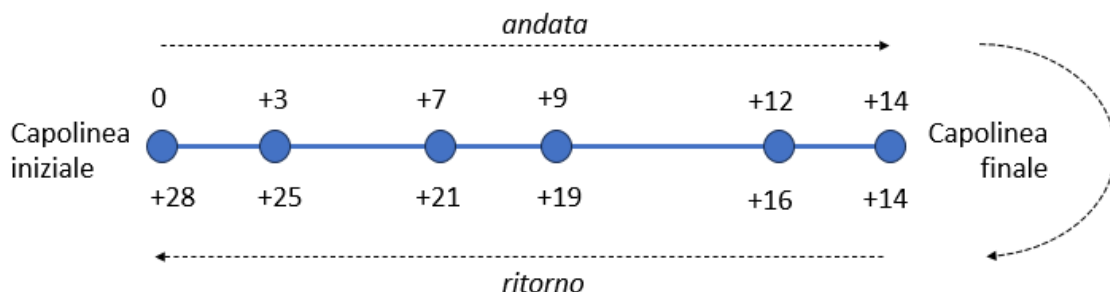
Nessuna linea effettua servizio a cavallo di mezzanotte: ogni linea effettua servizio da un certo orario iniziale (prima corsa), comunque non precedente le 00:00, a un certo orario finale (ultima corsa), comunque non successivo alle 23:59.

Per ipotesi, tutte le corse originano dal capolinea iniziale, effettuano l'intero tragitto fino al capolinea finale e poi – senza attesa – ripartono immediatamente per il capolinea iniziale. Pertanto, l'orario delle corse dal capolinea finale è traslato, rispetto al capolinea iniziale, di una durata pari alla lunghezza in minuti del tragitto e ogni corsa ritorna al capolinea iniziale dopo un tempo pari al *doppio* della durata del tragitto (v. figura).

La *frequenza naturale* del servizio, ossia quella che si avrebbe usando un solo bus che faccia avanti/indietro, è quindi pari al *doppio della durata del tragitto*. È possibile garantire corse più frequenti utilizzando più bus: in tal caso, la frequenza adottata dev'essere necessariamente *sottomultipla* della frequenza naturale.

ESEMPIO: una linea che faccia servizio al capolinea iniziale dalle 6:30 alle 21:26, con durata del tragitto fino al capolinea finale di 14 minuti, avrà corse "di andata" dal capolinea iniziale fra le 6:30 e le 21:26 e corse "di ritorno" dal capolinea finale dalle 6:44 alle 21:40, che torneranno al capolinea iniziale fra le 6:58 e le 21:54, rispettivamente.

La frequenza naturale del servizio è quindi di 28 minuti, corrispondente al servizio di un solo bus. Volendo garantire corse più frequenti, le frequenze possibili sono i sottomultipli di 28 e dunque 14' (usando due bus), 7' (usandone quattro), 4' (usandone sette) o 1' (usandone ventotto).



Questo modello di funzionamento richiede perciò il rispetto di due vincoli:

1. La durata del servizio (differenza fra ultima e prima corsa) dev'essere *multiplo pari* della durata del tragitto
2. La frequenza operativa del servizio dev'essere uguale o sottomultipla della *frequenza naturale* del tragitto

Nel caso sopra, la durata del servizio è di 14h56 = 896', che in effetti è multiplo pari della durata del tragitto (14'); inoltre, la frequenza naturale è 28', compatibile quindi con una frequenza operativa – ad esempio – di 7'.

Le linee sono descritte nei file di testo [Linee.txt](#), nel formato esplicitato più oltre.

TEMPO STIMATO PER SVOLGERE L'INTERO COMPITO:

2h15 – 3h

PARTE 1 – Modello dei dati: Punti 13

[TEMPO STIMATO: 75-90 minuti]

PARTE 2 – Persistenza: Punti 9

[TEMPO STIMATO: 35-50 minuti]

PARTE 3 – Grafica: Punti 8

[TEMPO STIMATO: 25-40 minuti]

JAVAFX – Parametri run configuration nei LAB

```
--module-path "C:\applicativi\moduli\javafx-sdk-19\lib"  
--add-modules javafx.controls
```

Cose da ricordare

- salva costantemente il tuo lavoro: l'informatica a volte può essere “subdolamente ostile”..
- in particolare: se ora compila e stai per fare modifiche, salva la versione attuale (non si sa mai)

Checklist di consegna

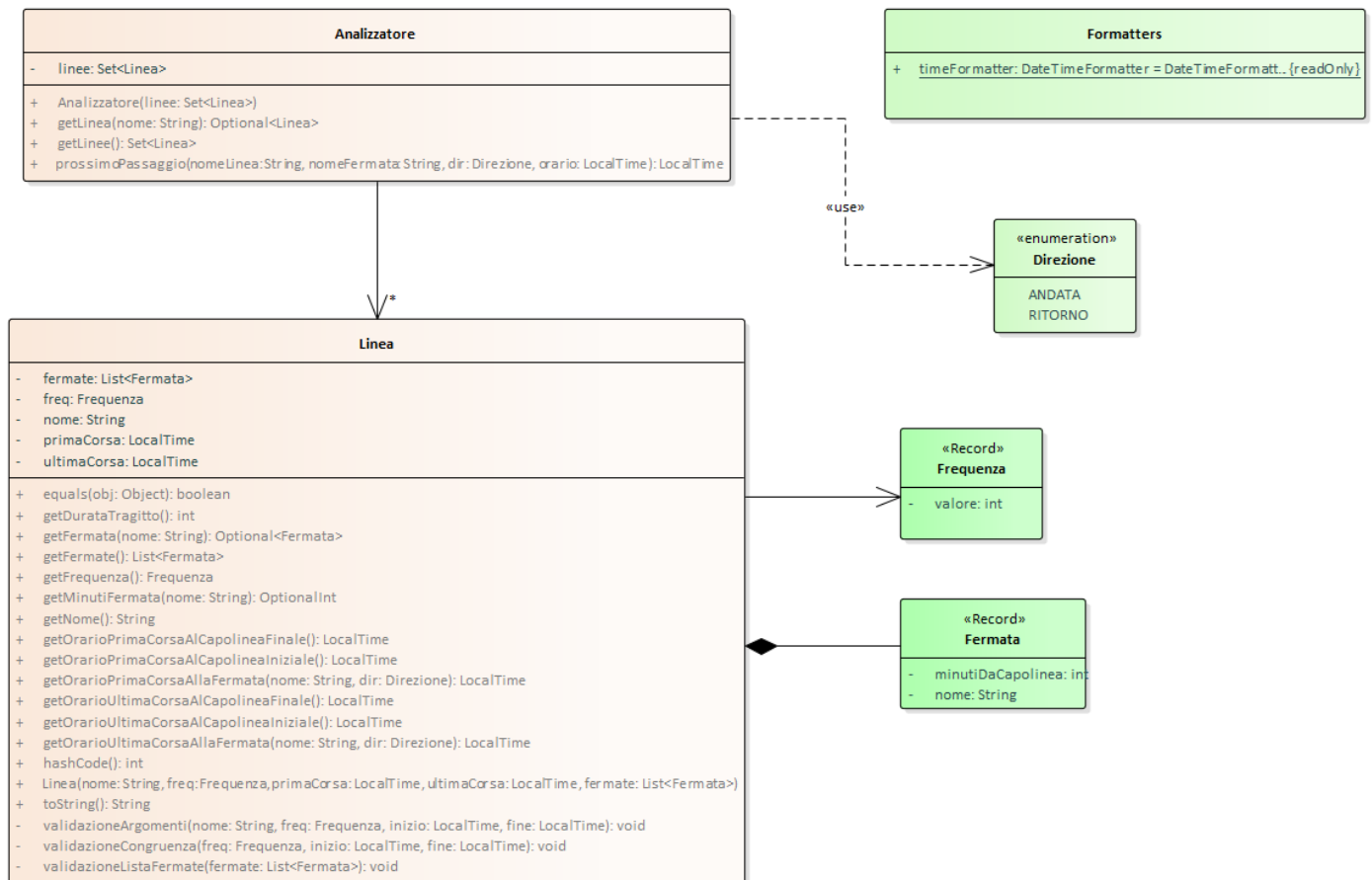
- Hai fatto un **JAR eseguibile**, che contenga cioè l'indicazione del main?
- Hai controllato che **si compili e ci sia tutto?** [NB: non includere il PDF del testo]
- Hai **rinominato** IL PROGETTO, lo ZIP e il JAR esattamente come richiesto?
- Hai **chiamato** la cartella del progetto esattamente come richiesto?
- **Hai fatto un unico file ZIP (NON .7z, rar o altri formati) contenente l'intero progetto?**
In particolare, ti sei assicurato di aver incluso tutti i file .java (e non solo i .class)?
- **Hai consegnato DUE file distinti, ossia lo ZIP col progetto e il JAR eseguibile?**
- Su EOL, hai **premuto** il tasto “CONFERMA” per inviare il tuo elaborato?

Parte 1 – Modello dei dati

(punti: 13)

Package: tram.model

[TEMPO STIMATO: 75-90 minuti]



SEMANTICA:

- L'enumerativo **Direzione** (fornito) definisce semplicemente le due costanti **ANDATA** e **RITORNO**
- Il record **Frequenza** (fornito) costituisce un *wrapper* per un valore intero al fine di garantire maggiore espressività rispetto al mero uso di un tipo primitivo: il suo costruttore verifica che il valore incapsulato sia compreso fra 1 e 60, estremi inclusi, lanciando **IllegalArgumentException** in caso contrario.
[NB: si ricorda che un record gode della generazione automatica di *equals*, *hashCode*, *toString* e di accessor di nome uguale agli argomenti, in questo caso quindi *valore*]
- Analogamente, il record **Fermata** (fornito) definisce una fermata come coppia (*nome*, *distanza temporale*), intendendosi con questo termine la distanza in minuti dal capolinea iniziale. Il costruttore verifica che il nome non sia nullo né blank e che il valore temporale incapsulato non sia negativo, lanciando **IllegalArgumentException** se necessario. La *toString* è definita in modo da restituire soltanto il nome della fermata.
- La classe **Formatters** (fornita) definisce il formattatore (*timeFormatter*) per formattare e fare parsing di orari secondo lo standard italiano.
- La classe **Linea** (da completare nella sola validazione) descrive una linea intesa come entità dotata di nome, orario di servizio (ossia orario della prima e dell'ultima corsa al capolinea iniziale), frequenza operativa ed elenco di fermate. Sono forniti già implementati tutti i metodi pubblici: rimane da realizzare solo quello, privato, utilizzato dal costruttore, che effettua le verifiche di congruenza relative al rispetto dei vincoli specificati dal Dominio del Problema (*validazioneCongruenza*). Sono quindi forniti già implementati:
 - equals*, *hashCode*, *toString* tutte definite in modo da considerare soltanto il nome della linea

- *getNome*, *getFrequenza*, *getOrarioPrimaCorsaAlCapolineaIniziale*, *getOrarioUltimaCorsaAlCapolineaIniziale*, *getFermate* che restituiscono semplicemente gli argomenti ricevuti dal costruttore
- *getOrarioPrimaCorsaAlCapolineaFinale*, *getOrarioUltimaCorsaAlCapolineaFinale*, che calcolano e restituiscono i corrispondenti orari al capolinea finale, tenuto conto della durata del tragitto
- *getOrarioPrimaCorsaAllaFermata(String nome, Direzione dir)*, *getOrarioUltimaCorsaAllaFermata(String nome, Direzione dir)*, che calcolano e restituiscono gli orari della prima/ultima corsa alla fermata specificata, tenuto conto della direzione del viaggio (corsa di andata o di ritorno)
- *getDurataTragitto*, che calcola la durata del tragitto intesa come differenza fra gli orari della prima e dell'ultima corsa al capolinea iniziale
- *getFermata(String nome)*, che cerca e restituisce, se esiste, la fermata di nome uguale a quello passato come argomento: per questa ragione il tipo restituito è un **Optional<Fermata>**
- *getMinutiFermata(String nome)*, che, analogamente a sopra, cerca, se esiste, la fermata di nome uguale a quello passato come argomento e nel caso restituisce la sua distanza in minuti dal capolinea iniziale: per questa ragione il tipo restituito è un **OptionalInt**
- i due metodi privati *validazioneArgomenti* e *validazioneListaFermate*, che verificano, rispettivamente:
 - *validazioneArgomenti*, che gli argomenti siano non nulli (nel caso di stringhe, anche non-blank) e che gli orari di inizio e fine siano coerenti (ossia, il secondo non preceda il primo)
 - *validazioneListaFermate*, che non vi siano nomi di fermate duplicati, che non vi siano due fermate con la stessa distanza temporale dal capolinea iniziale, e che vi sia sempre una fermata con distanza temporale 0 dal capolinea iniziale (il capolinea iniziale stesso)

Deve invece essere implementato:

- *validazioneCongruenza*, che deve verificare sia che la durata del servizio sia coerente con la durata del tragitto (vincolo 1), sia che quest'ultima sia coerente con la frequenza del servizio (vincolo 2), lanciando **IllegalArgumentException** con opportuno messaggio d'errore in caso contrario

f) La classe **Analizzatore** (da completare) costituisce il componente software fondamentale, in grado di calcolare l'orario del prossimo passaggio di una qualunque linea a una qualunque fermata. A tal fine:

- Il costruttore riceve il set di linee che costituiscono la rete, recuperabile tramite l'accessor *getLinee*; l'ulteriore accessor *getLinea(String nome)* restituisce, se esiste, la **Linea** di nome uguale all'argomento fornito: per questa ragione il tipo restituito è un **Optional<Linea>**
- il metodo *prossimoPassaggio* (da implementare) calcola l'orario della prossima corsa *di orario pari o successivo* a quello specificato come (quarto) argomento. Più in dettaglio, esso ricerca, nella linea ricevuta come primo argomento, la fermata ricevuta come secondo argomento e calcola l'orario del prossimo passaggio nella direzione (andata o ritorno) specificata dal terzo argomento. Il metodo deve preventivamente verificare, lanciando **IllegalArgumentException** in caso di violazione:
 - che gli argomenti non siano nulli, o, nel caso di stringhe, anche blank
 - che una *linea* di nome uguale a quello specificato esista realmente

Non occorre invece verificare che una *fermata* di nome uguale a quello specificato esista realmente perché tale controllo è già svolto nei metodi *getOrarioXXX* di **Linea**, che lanciano anch'essi **IllegalArgumentException** in tale evenienza.

Successivamente, il metodo agisce come segue:

- calcola l'orario di servizio (prima/ultima corsa) alla fermata richiesta *nella direzione indicata*
- se l'orario richiesto è esterno all'orario di servizio così calcolato, restituisce l'orario della prima corsa (che si svolgerà, sottinteso, nel giorno successivo)

- se invece l'orario richiesto è ricompreso nell'orario di servizio odierno, calcola, tenendo conto della frequenza del servizio, l'orario della prima corsa di orario uguale o successivo a quello richiesto, e lo restituisce come risultato [NB: vedere i test nel codice per moltissimi esempi]

SUGGERIMENTO: calcolare quante corse ci stanno nell'intervallo temporale fra l'orario richiesto e quello della prima corsa, e sottrarre la durata da tale intervallo...

ESEMPIO:

- linea operante al capolinea iniziale dalle 06:30 alle 21:26, durata tragitto 14', frequenza 7';
- fermata richiesta "Giardino", a 10' dal capolinea iniziale (e quindi a 4' dal capolinea finale)

ESEMPI DI RICHIESTE:

- possibili orari richiesti in ANDATA: 06:30, 13:20, 13:26, 20:45, 21:45
- possibili orari richiesti in RITORNO: 06:30, 19:59, 20:00, 20:50, 21:50

COME OPERA IL METODO:

- preliminarmente calcola l'orario di servizio a tale fermata in ANDATA o RITORNO, secondo la direzione richiesta: nel primo caso otterrà 06:40-21:36, nel secondo 06:48-21:44
- confronta quindi l'orario richiesto per la corsa con l'orario di servizio sopra calcolato
- per orari esterni all'orario di servizio (quindi nell'esempio 06:30, 21:45 in andata, 06:30, 21:50 in ritorno) risponde indicando la prima corsa (eventualmente, da intendersi del giorno successivo per gli orari a tarda sera), ovvero 06:40 in andata o 06:48 in ritorno
- per quelli interni, calcola la prossima corsa a questa fermata tenendo conto della frequenza del servizio, ottenendo in questo caso, rispettivamente, per l'andata 13:26 (sia per la richiesta delle 13:20 che per quella delle 13:26), 20:47 (per quella delle 20:45), per il ritorno 19:59, 20:06, 20:55.

Parte 2 – Persistenza

Package: tram.persistence

(punti: 9)

[TEMPO STIMATO: 35-50 minuti]

Le linee di tram sono elencate nel file di testo `Linee.txt`: non si sa quante siano, ma ognuna è descritta su due righe, entrambe articolate in elementi separati da virgole. Più precisamente:

- la prima riga specifica il nome della linea, la frequenza e gli orari di servizio al capolinea iniziale: il nome è preceduto dalla parola "Linea" seguita da spazi, la frequenza è un valore intero non negativo e non superiore a 60, ricompreso fra le due parole "frequenza" e "minuti", separate da spazi, mentre gli orari della prima e dell'ultima corsa al capolinea iniziale sono espressi nel formato italiano short HH:MM, separati fra loro da un trattino (senza spazi intermedi)
- la seconda riga elenca invece le varie fermate, indicando per ciascuna:
 - il nome, eventualmente seguito da spazi per comodità di lettura
 - la distanza temporale in minuti dal capolinea iniziale (un intero non negativo), indicata fra parentesi tonde.

NB: le fermate sono elencate senza alcun particolare ordine, ma devono avere, in ciascuna linea, nomi distinti e distanze temporali tutte distinte: ciò è verificato, come già spiegato, dal costruttore di **Linea**. È ovviamente invece senz'altro possibile che una fermata di egual nome serva più linee (con distanze temporali probabilmente diverse da una linea all'altra).

ESEMPIO

Linea Rossa, frequenza 5 minuti, 06:30-21:30

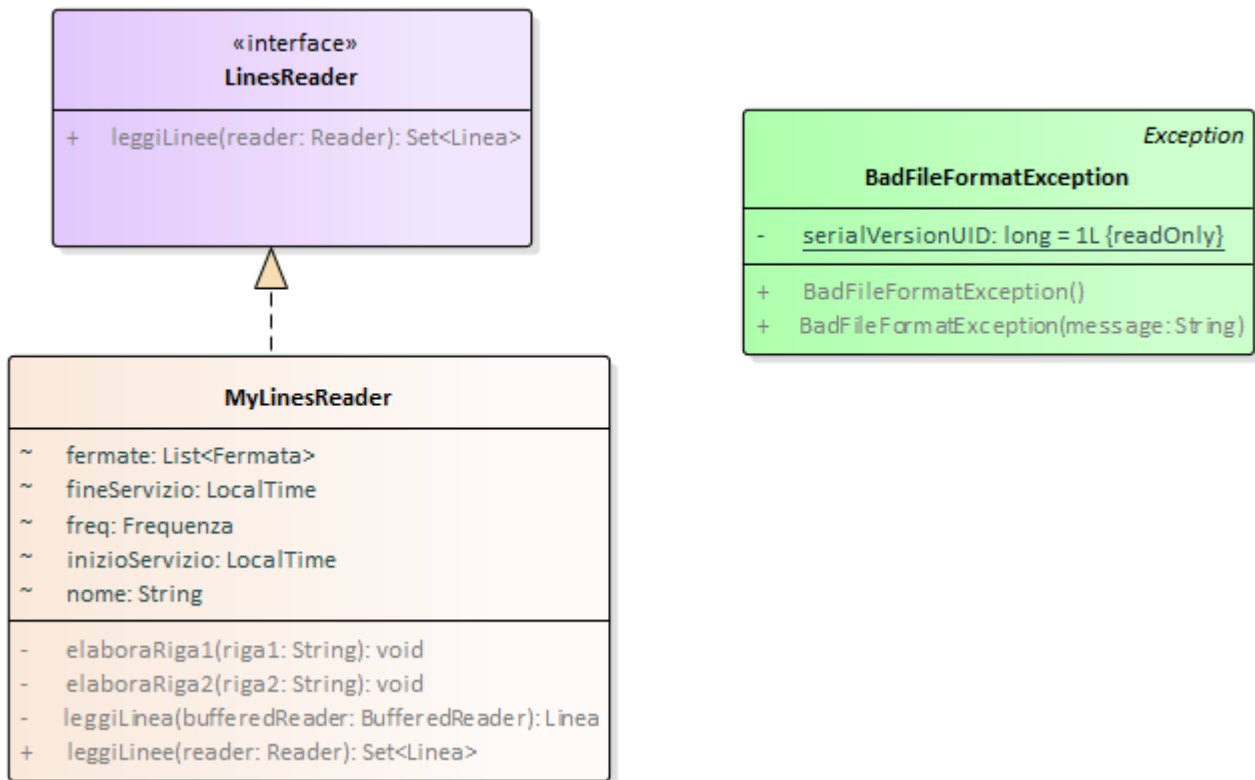
Piazza Celestini (0), via Amendola (3), ..., porta Fiorentina (25)

Linea Verde, frequenza 8 minuti, 06:00-22:00

Porta Romana (0), via Garibaldi (4), via Morandi (7), ..., Stazione FS (24)

Linea Gialla, frequenza 10 minuti, 06:00-23:20

Autostazione (0), via dei Frati (2), via delle Querce (5), ..., largo Nuvolari (20)



SEMANTICA:

- a) L'interfaccia **LinesReader** (fornita) dichiara il metodo **leggiLinee** che carica da un apposito Reader (già aperto) i dati necessari e restituisce un **Set** di **Linea** opportunamente popolato. Devono essere lanciate:
- **IllegalArgumentException** con opportuno messaggio d'errore in caso di argomento (reader) nullo;
 - **BadFormatException** con messaggio d'errore appropriato in caso di problemi nel formato del file (mancanza/eccesso di elementi, errori nel formato dei numeri o degli orari, etc.)
 - una **IOException** in caso di altri problemi di I/O.
- b) La classe **LinesReader** (da completare) implementa **LinesReader** secondo le specifiche sopra descritte. Il metodo pubblico **leggiLinee** è fornito già implementato e si avvale del metodo ausiliario privato **leggiLinea** (anch'esso fornito) per leggere dal **BufferedReader** una singola **Linea**. A sua volta, quest'ultimo delega l'elaborazione delle due righe ad altrettanti metodi ausiliari privati **elaboraRiga1** ed **elaboraRiga2** (da implementare).

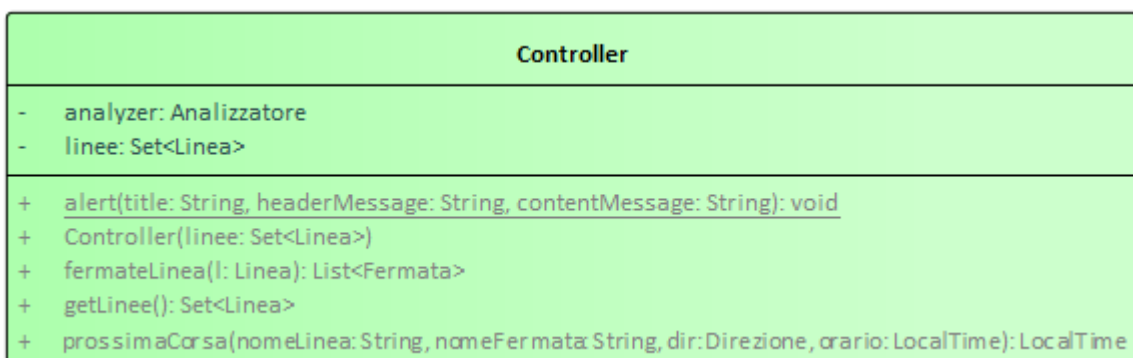
Parte 3

(punti: 8)

Package: *tram.controller*

(punti 0)

Il Controller funge semplicemente da front-end verso l'analizzatore, ed è quindi organizzato come segue:



SEMANTICA:

La classe **Controller** (fornita) riceve in fase di costruzione l'insieme di linee della rete, che usa poi – previa validazione di coerenza – per costruire internamente l'analizzatore, usato poi per rispondere alle interrogazioni dell'utente.

I metodi si limitano a richiamare quelli di **Analizzatore**, talora con minime differenze di naming. In particolare:

- **getLinee** restituisce l'insieme delle linee fornito al costruttore
- **fermateLinea** restituisce la lista delle fermate di una data Linea: se essa non esiste, lancia **IllegalArgument-Exception** con apposito messaggio.
- **prossimaCorsa** è un semplice front-end verso il metodo **prossimoPassaggio** di **Analizzatore**

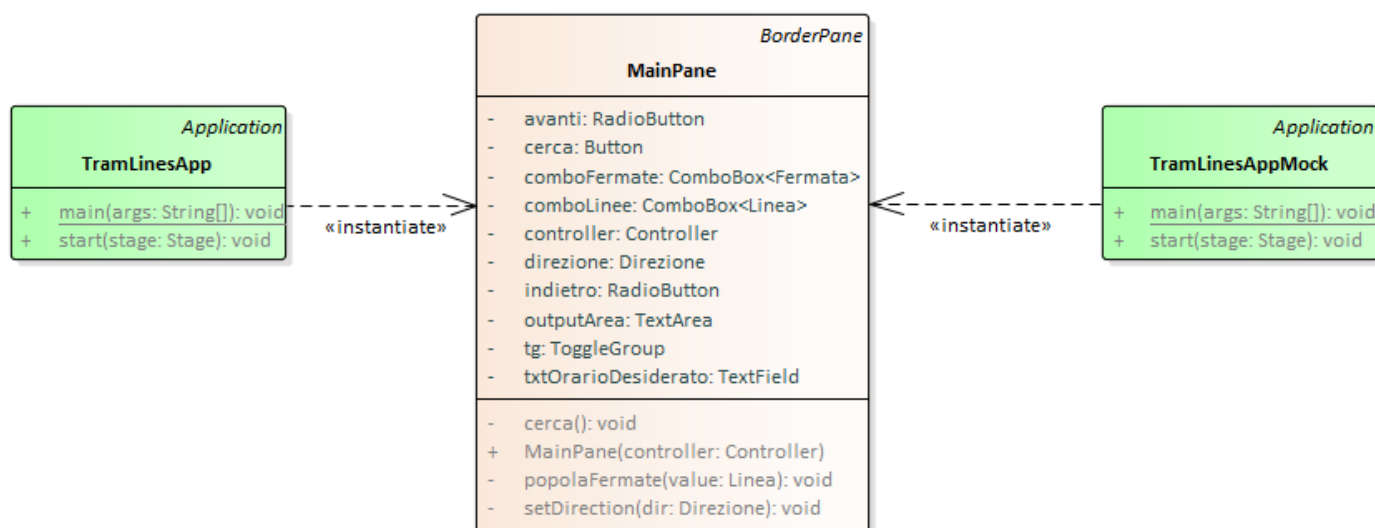
Infine, il metodo statico **alert**, utilizzabile anche dal **MainPane**, consente di far comparire all'utente, ove occorra, una finestra di dialogo con opportuno messaggio d'errore.

Package: tram.ui

[TEMPO STIMATO: 25-40 minuti] (punti 8)

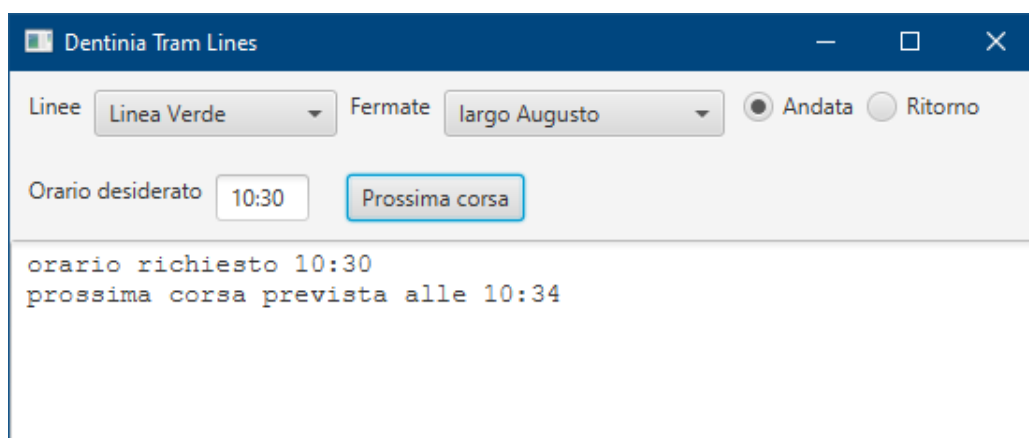
La classe **TramLinesApp** (fornita) costituisce l'applicazione JavaFX che si occupa di aprire i file, creare il controller e incorporare il **MainPane**. Per consentire di collaudare la GUI anche in assenza / in caso di malfunzionamento della parte di persistenza, è possibile avviare l'applicazione mediante la classe **TramLinesAppMock** (NB: essa utilizza linee diverse da quelle elencate nel file)

L'interfaccia utente è illustrata nelle figure seguenti e segue il modello sotto illustrato:



L'interfaccia grafica si presenta come segue:

- in alto, le due combo con l'elenco delle linee e delle fermate, seguite dai radiobutton per la scelta della direzione
- al centro, il campo di testo in cui scrivere l'orario desiderato e, a fianco, il bottone per attivare la ricerca
- in basso, un'area di testo in cui vengono mostrati i risultati.



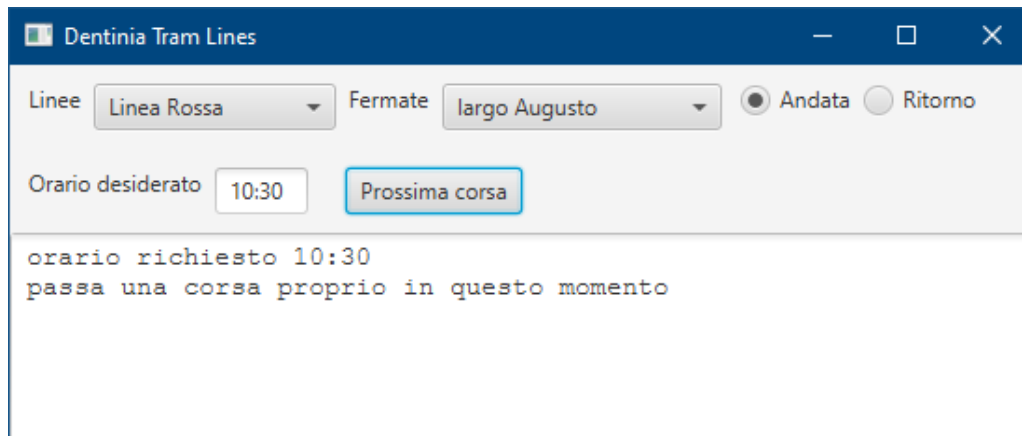
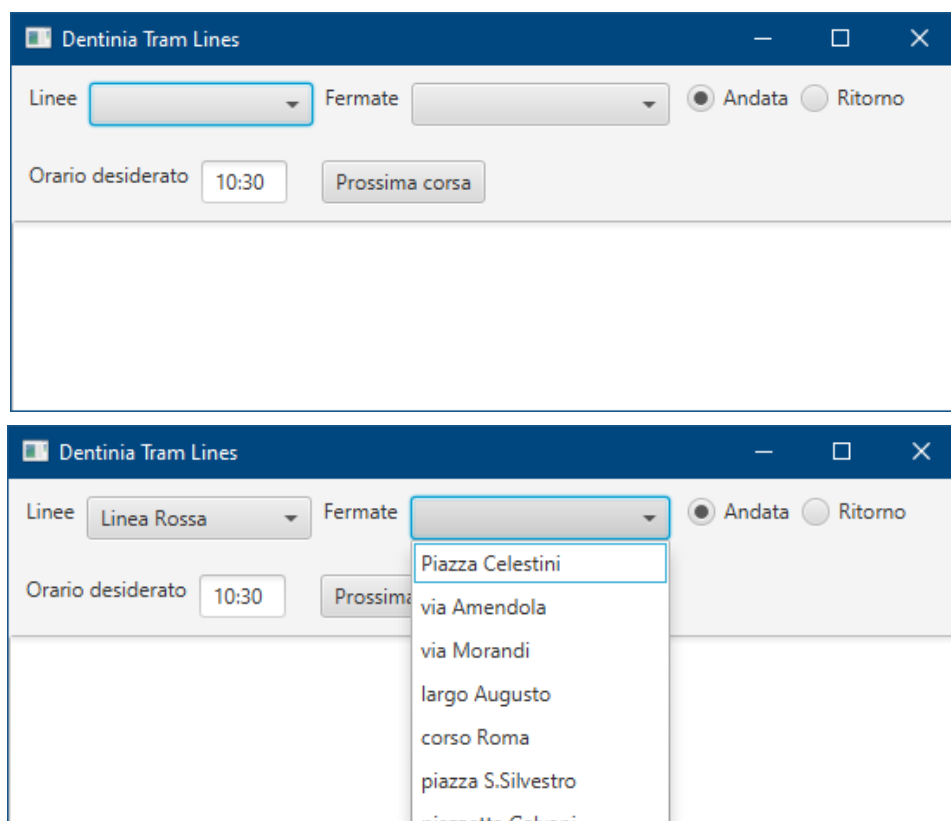


Fig. 1: vista generale della GUI. In alto le combo per scegliere linea e fermata, e i radiobutton per la scelta della direzione; a seguire, il campo per inserire l'orario e il pulsante per attivare la ricerca. In basso, l'area risultati.

Comportamento:

- preliminarmente l'utente deve selezionare la linea di suo interesse dalla combo in alto a sinistra: ciò determina il caricamento, nella combo a fianco, delle fermate della linea prescelta
- l'utente deve poi scegliere la fermata di suo interesse dalla seconda combo (non accade ancora nulla) e selezionare la direzione della corsa di suo interesse tramite gli appositi radiobutton (anche qui non accade nulla)
- a questo punto l'utente deve digitare nel campo di testo l'orario desiderato, nel formato italiano HH:MM (non accade ancora nulla) e infine premere il pulsante "Prossima corsa"
- la pressione del pulsante attiva la ricerca e determina l'apparizione del risultato nell'area di testo sottostante.

Il testo visualizzato deve riportare, come nell'esempio, sia l'orario richiesto, sia quello della prossima corsa: nel caso essa sia proprio all'orario indicato, ciò va adeguatamente evidenziato con messaggistica ad hoc.



Dentinia Tram Lines

Linee: Linea Rossa Fermate: corso Roma ☒ Andata ☐ Ritorno

Orario desiderato: 10:30 Prossima corsa

orario richiesto 10:30
prossima corsa prevista alle 10:32

Fig. 2: in alto, la situazione iniziale; al centro, la GUI dopo aver scelto la linea: la seconda combo è stata popolata con l'elenco delle corrispondenti fermate. In basso, dopo aver scelto la fermata (nonché direzione e orario), la pressione del pulsante "Prossima corsa" determina la produzione nell'area di output del risultato richiesto.

Dentinia Tram Lines

Linee: Linea Rossa Fermate: corso Roma ☒ Andata ☐ Ritorno

Orario desiderato: 22:30 Prossima corsa

orario richiesto 22:30
prossima corsa prevista alle 06:42

Dentinia Tram Lines

Linee: Linea Rossa Fermate: corso Roma ☐ Andata ☒ Ritorno

Orario desiderato: 22:30 Prossima corsa

orario richiesto 22:30
prossima corsa prevista alle 07:08

Fig. 3: sopra: se l'orario indicato è successivo al termine del servizio, viene proposta la prima corsa del giorno successivo. Sotto: stesso scenario ma relativamente alla corsa di ritorno..

Linea non selezionata

Occorre prima selezionare la linea

Selezionare la linea desiderata dalla combo in alto a sinistra

OK

Fermata non selezionata

Occorre selezionare la fermata

Selezionare la fermata desiderata dalla combo in alto al centro

OK

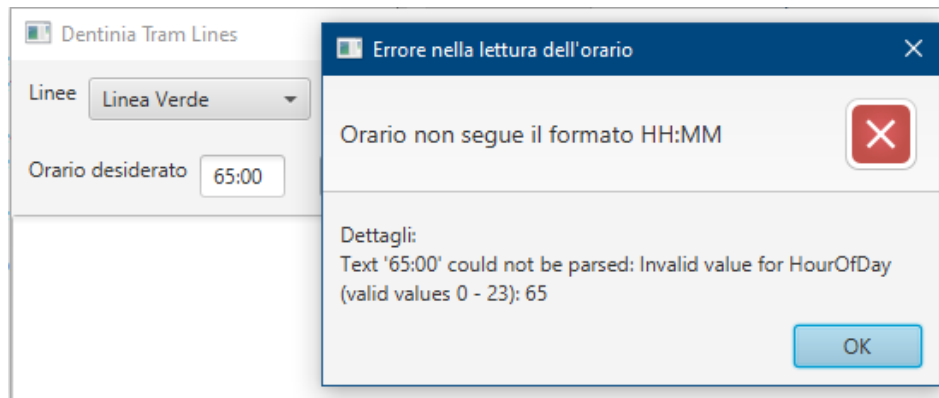


Fig. 4: in alto: messaggi d’errore nel caso si attivi la ricerca rispettivamente senza aver prima selezionato la linea (a sinistra) o la fermata (a destra); sotto, nel caso in cui l’orario sia scorretto o non rispetti il formato richiesto.

Il MainPane è fornito *parzialmente realizzato*: è presente quasi tutta la parte strutturale ed è già implementata la gran parte dei metodi. Rimangono da realizzare:

- 1) il popolamento iniziale della combo delle linee
- 2) il relativo gestore degli eventi (metodo privato *popolaFermate*) che popola l’altra combo
- 3) il metodo privato *cerca*, che gestisce il pulsante “Prossima corsa”

In particolare, quest’ultimo deve:

- preliminarmente, recuperare la linea selezionata dalla combo linee: se questa è nulla, si emette un messaggio d’errore (tramite il metodo *alert* del **Controller**) e si ritorna senza fare nulla
- recuperare la fermata selezionata dalla combo fermate: anche in questo caso, se essa è nulla si deve emettere un messaggio d’errore (tramite il metodo *alert* del **Controller**) e ritornare senza fare nulla
- recuperare l’orario dal campo di testo apposito e farne il parsing: ancora una volta, in caso di errore occorre emettere un apposito messaggio (tramite il metodo *alert* del **Controller**) e ritornare
- infine, tramite il controller, provvedere al calcolo dell’orario della prossima corsa e sintetizzare gli appositi messaggi da emettere sull’area di testo.

Cose da ricordare

- salva costantemente il tuo lavoro: l’informatica a volte può essere “subdolamente ostile”..
- in particolare: se ora compila e stai per fare modifiche, salva la versione attuale (non si sa mai)

Checklist di consegna

- Hai fatto un **JAR eseguibile**, che contenga cioè l’indicazione del main?
- Hai controllato che **si compili e ci sia tutto?** [NB: non includere il PDF del testo]
- Hai **rinominato** IL PROGETTO, lo ZIP e il JAR esattamente come richiesto?
- Hai **chiamato** la cartella del progetto esattamente come richiesto?
- **Hai fatto un unico file ZIP (NON .7z, rar o altri formati) contenente l’intero progetto?**
In particolare, ti sei assicurato di aver incluso tutti i file .java (e non solo i .class)?
- **Hai consegnato DUE file distinti, ossia lo ZIP col progetto e il JAR eseguibile?**
- Su EOL, hai **premuto** il tasto “CONFERMA” per inviare il tuo elaborato?